

NTU Timetable App

Table of Contents

Introduction.....	1
Introduction to project	1
Purpose of the Report	1
Aim and Objectives	1
Scope of the Project.....	2
Methodology	2
Planning	3
Requirement Analysis	3
Scheduling.....	4
System Design.....	5
Functional Analysis	5
Use Case Diagram	5
State-Transition Diagram.....	15
Structural Analysis	16
Class diagram	16
Entity Relationship Diagram (ERD).....	17

Introduction

Introduction to project

Timetabling plays a vital role in the academic management system at Nottingham Trent University (NTU), ensuring that lectures, rooms, and student groups are coordinated effectively across various modules and programmes. The official NTU Student Dashboard provides a comprehensive interface where students can access their schedules, view room allocations, and plan their academic week. However, this project focuses on creating a simplified, console-based prototype of that system — capturing the core functionality while maintaining clarity and ease of use.

Purpose of the Report

The purpose of this report is to document the design, development, and evaluation of a prototype timetabling system tailored for NTU. It outlines the technical decisions, system features, and design methodologies used to create a functional backend application that supports both administrative and student-facing features.

Aim and Objectives

The aim of this project is to design and implement a console-based timetabling system for NTU that can generate, manage, and retrieve class schedules efficiently.

Key objectives include:

- To enable **administrators** to manage modules, student groups, lecturers, sessions, and timetable assignments.
- To allow **students** to view and search for their weekly schedules.
- To implement conflict detection and resolution for overlapping sessions.
- To apply object-oriented programming concepts to ensure modular, maintainable code.
- To test and evaluate the system against specified functional and non-functional requirements.

Scope of the Project

While the full NTU Student Dashboard encompasses various integrated services, this project is intentionally limited in scope to focus on the essential features of timetable management. These include:

- A user interface for both Admin and Student roles through the console.
- Core functionalities such as adding modules, assigning lecturers, managing group schedules, and viewing timetables.
- Simplified data storage using in-memory data structures, with optional file-based persistence.
- Exclusion of advanced features such as real-time notifications, GUI interfaces, and integration with NTU's live databases.

This project serves as a **foundation for future development**, offering a scalable backend system that can potentially be expanded into a more complete and user-friendly platform.

Methodology

I will be using scrum methodology for this project as it provides me the following benefits:

- Adaptability to requirements and challenges.
- Clear milestones and Time Management.
- Regular reviewing.
- Flexible Risk Mitigation.
- Focus on incremental progress.

Planning

Requirement Analysis

Functional Requirements:

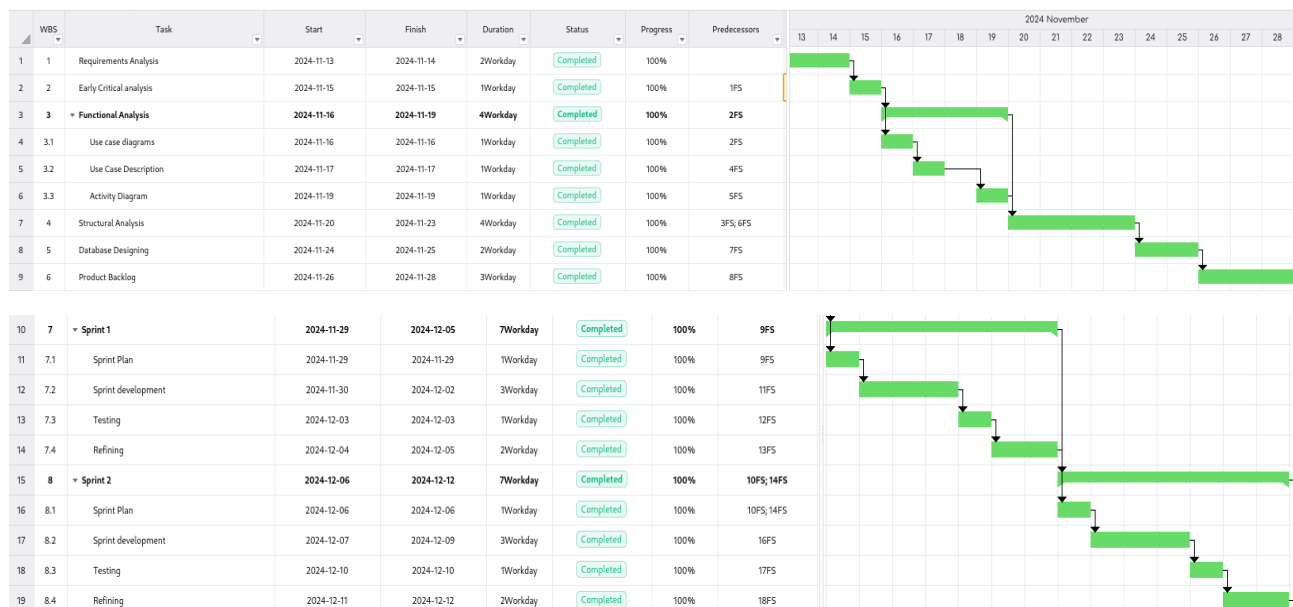
Index	User	Requirement	Implications	Priority
1	All users	Login/Logout	Able to login/logout of the system	Must
2		Display commands	View the commands that are applicable/available to the user	Should
3	Student	View current week's timetable	Able to view the ongoing week's timetable	Must
4		Search timetable by week number	Search and view timetable for a given week number	Must
5		Display weeks and their period	Print out the week numbers and their dates when search by week is called, i.e., week 9: 25/09/2024 - 02/10/2024	Should
6		Search timetable by room no.	Search and view sessions in a given room no. in the ongoing week	Should
7		Search timetable by module	Search and view sessions for a given module in the ongoing week	Should
8		Search timetable by lecturer	Search and view sessions with a given lecturer in the ongoing week	Should
9		Export timetable as csv/xlsx (Excel) file	Able to export the current screen into Excel file	Must
10	Lecturer	View current week's timetable	Able to view the ongoing week's timetable	Must
11		Search timetable by week number	Search and view timetable for a given week number	Must
12		Display weeks and their periods	Print out the week numbers and their dates when search by week is called, i.e., week 9: 25/09/2024 - 02/10/2024	Should
13		Search timetable by room no.	Search and view sessions in a given room no. in the ongoing week	Should
14		Search timetable by modules	Search and view sessions for a given module in the ongoing week	Should
15		Search timetable for a student	Search and view sessions for a given student in the ongoing week	Should
16		Export timetable as csv/xlsx (Excel) file	Able to export the current screen into Excel file	Must
17	Admin	Add/Delete modules	Able to add or delete a module	Must
18		Modify modules	Modify existing modules (Name, year, etc)	Must
19		Add/Delete student groups	Add or delete any student groups (to modules)	Must
20		Modify student groups	Modify existing student groups	Must
21		Define sessions	Define and set up sessions (lectures, labs, etc).	Must
22		Modify sessions	Modify existing sessions (time, location, modules, etc)	Must
23		Register students	Register students to the system	Must
24		Assign students to groups	Assign students to student groups	Must
25		Register lecturers	Register lecturers to the system	Must
26		Assign lecturers to sessions	Assign lecturers to existing sessions	Must

27		Create timetable for each week	Create timetable by week number for each week from week 1 and week 53	Must
28		Update timetable for each week	Update timetable by week number for each week	Must
29		Search for any timetable conflicts	Search if any timetable conflicts exist, and display if they do	Should

Non-Functional Requirements:

1. Student should only be able to view their own timetable and not anyone else's.
2. The application should operate on Windows, MacOS and Linux.
3. The application must ask administrator for a confirmation before making the changes.
4. The application must raise an error if the user does not exist in the database or if they input wrong password.
5. The application should be memory and time efficient.
6. The application should create a log of actions/changes made by the administrator in a session which can be accessed by a command.
7. The application should clear the console for every display timetable command.

Scheduling



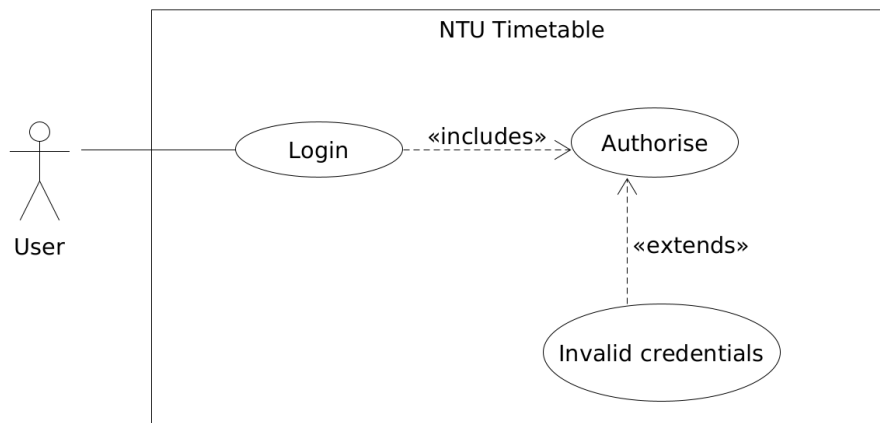


System Design

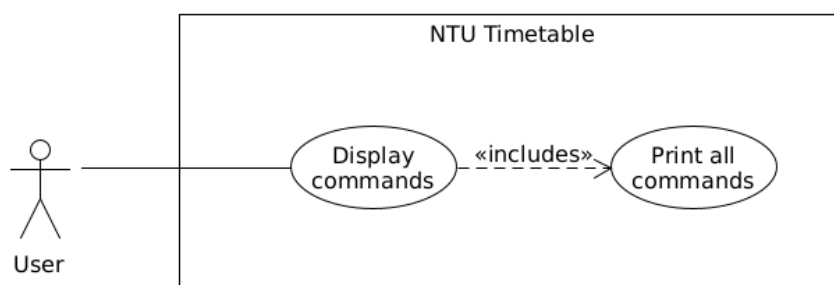
Functional Analysis

Use Case Diagram

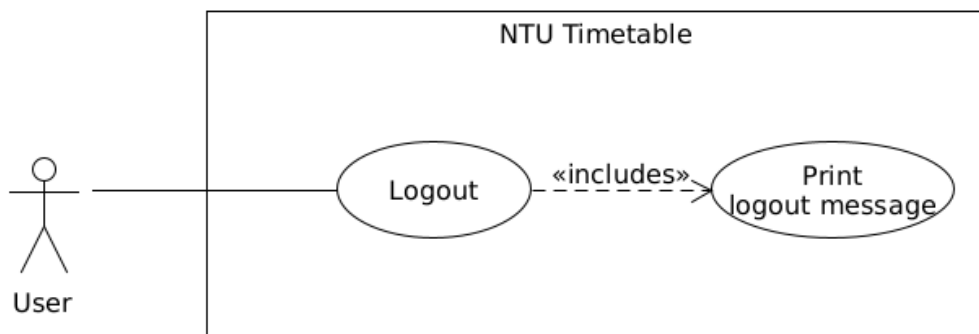
- All Users:
 - Login: User logs into the system with their credentials.



- Display Commands: To display all applicable commands to that user.



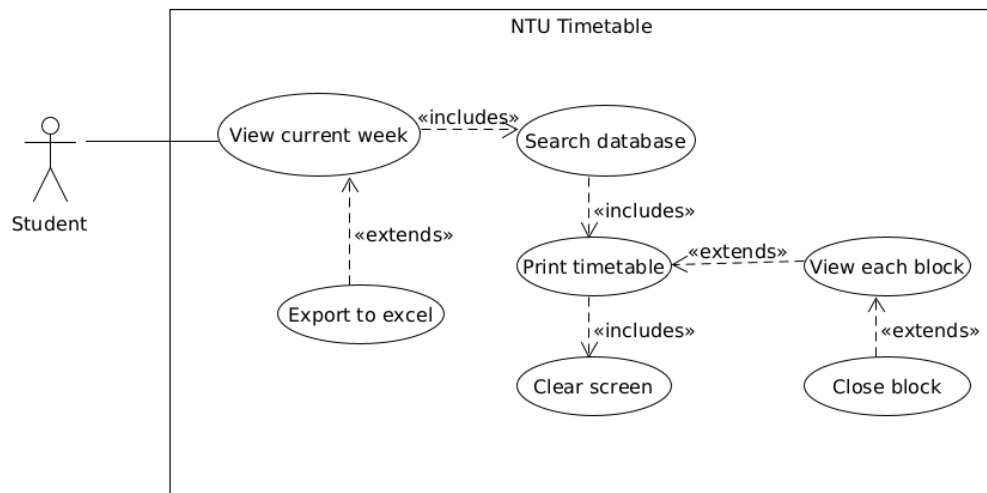
- Logout: Users logs out of the system



- Student:

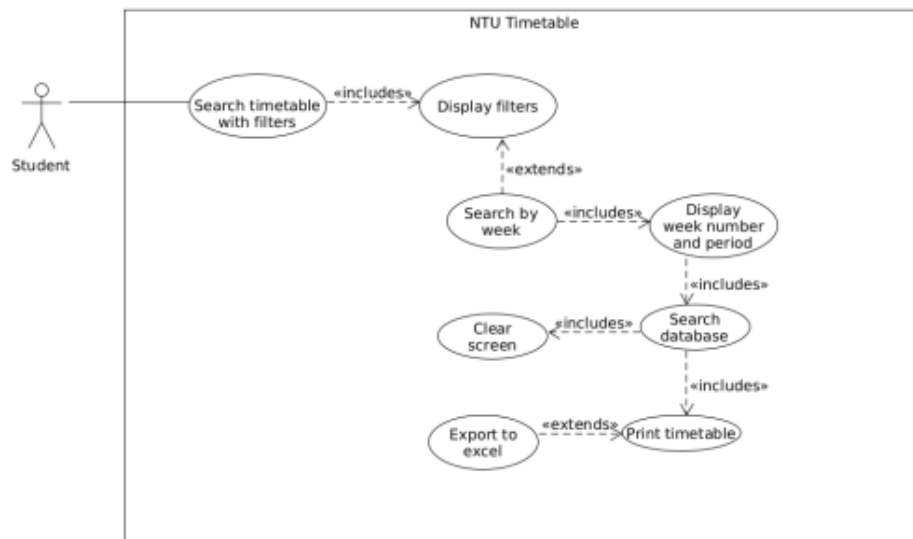
- View current week's timetable:

Student wants to quickly check the current week's timetable



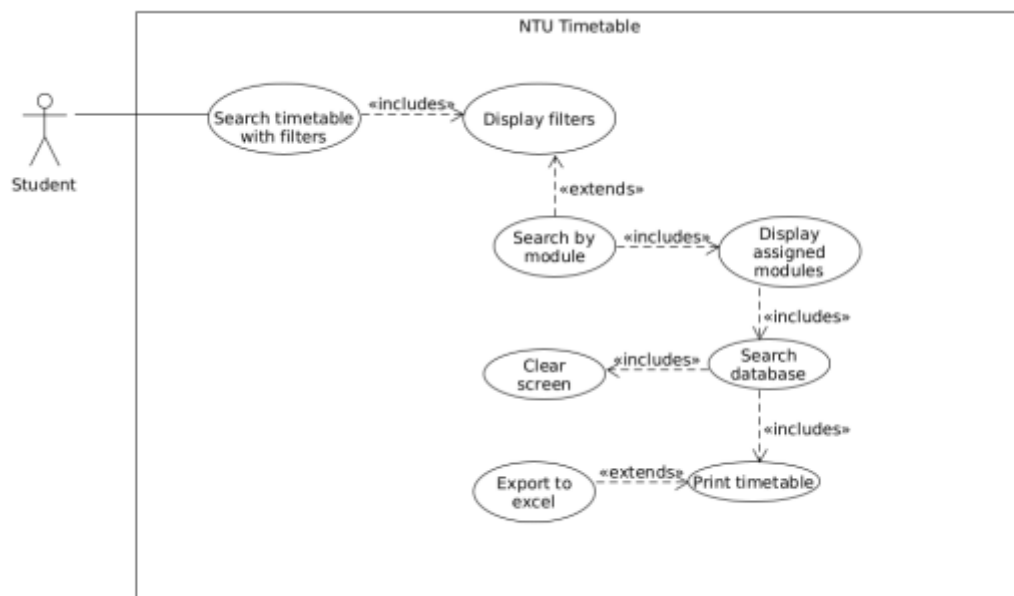
- Search timetable by week:

Student wants to take a look at any upcoming week's timetable



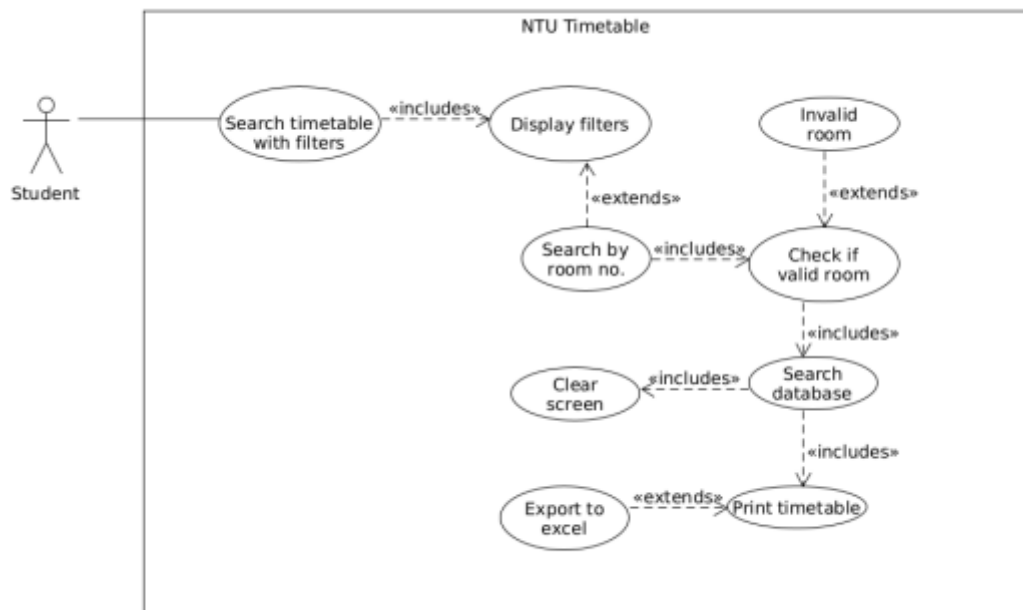
- Search timetable by module:

Student wants to see the sessions they have for a specific module in current week



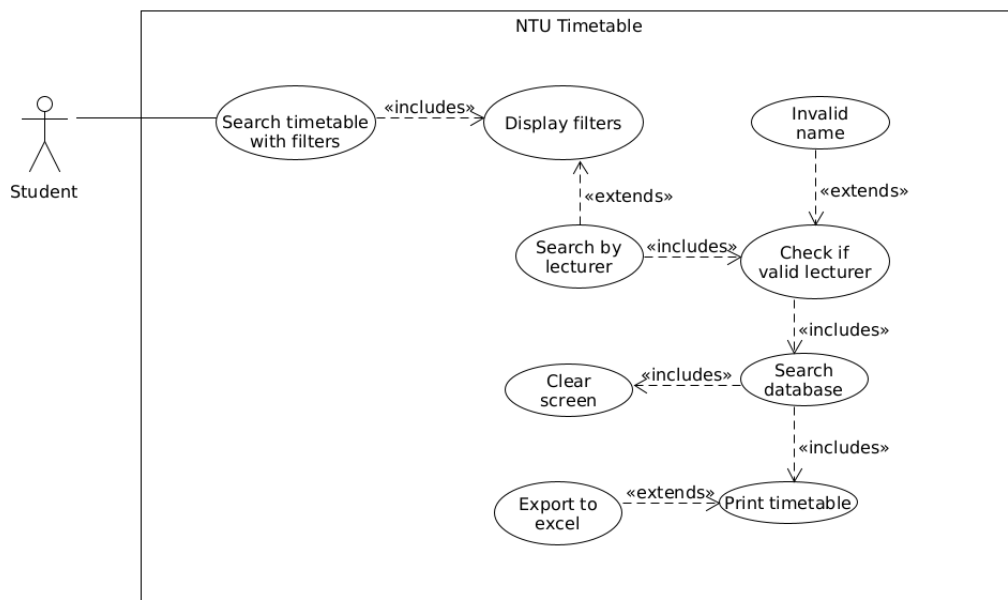
- Search timetable by room number:

Student wants to see when and what sessions do they have in a specific room in the current week



- Search timetable by lecturer/tutor:

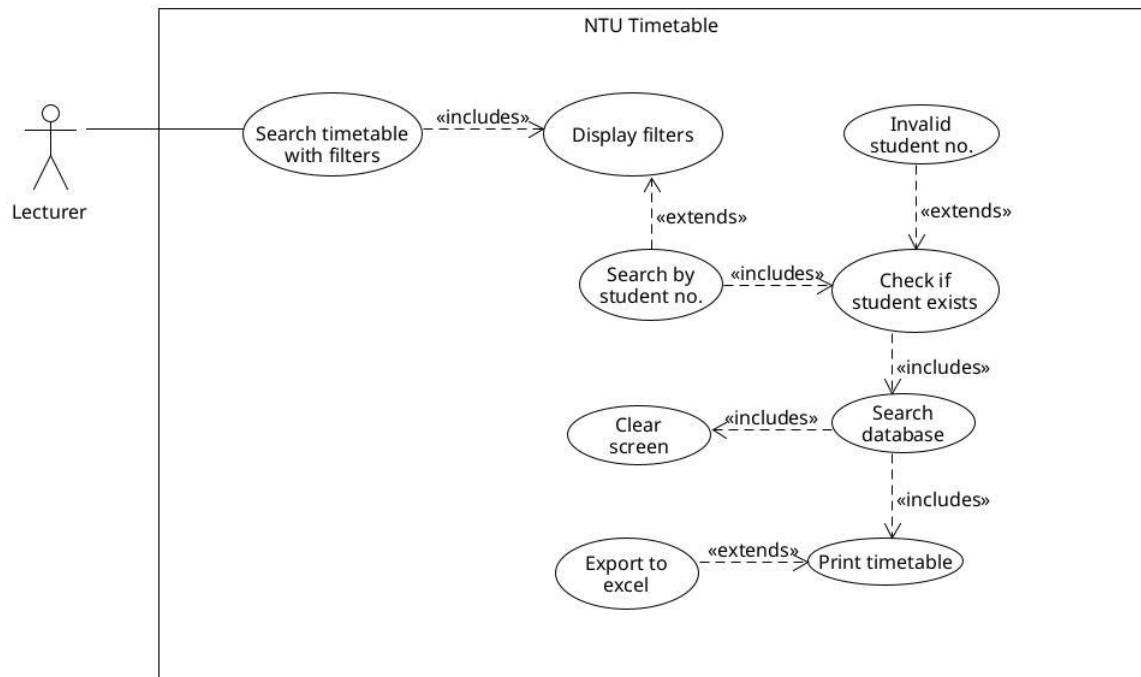
Student wants to view what sessions do they have with a specific tutor in the current week



- Lecturer

- Search timetable by student no.:

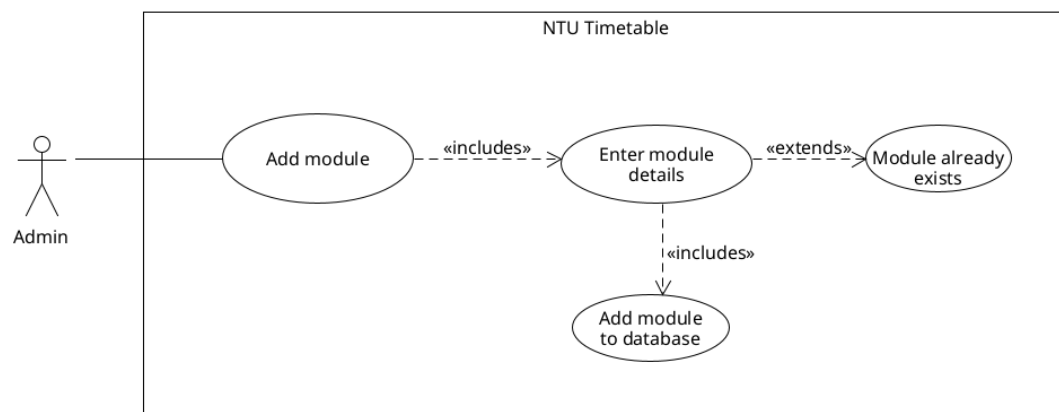
Lecturer wants to view a student's timetable in the ongoing week



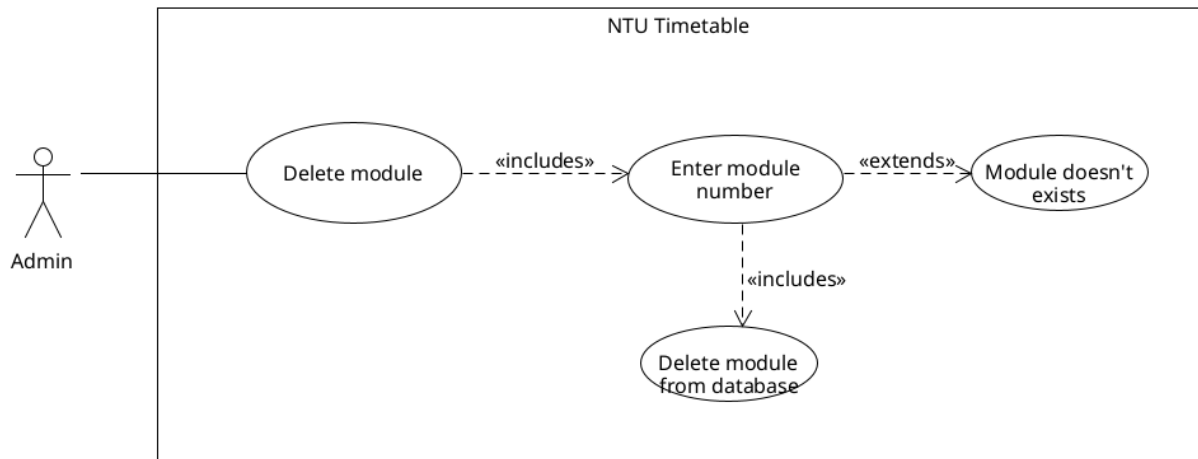
- Admin:

- Add module:

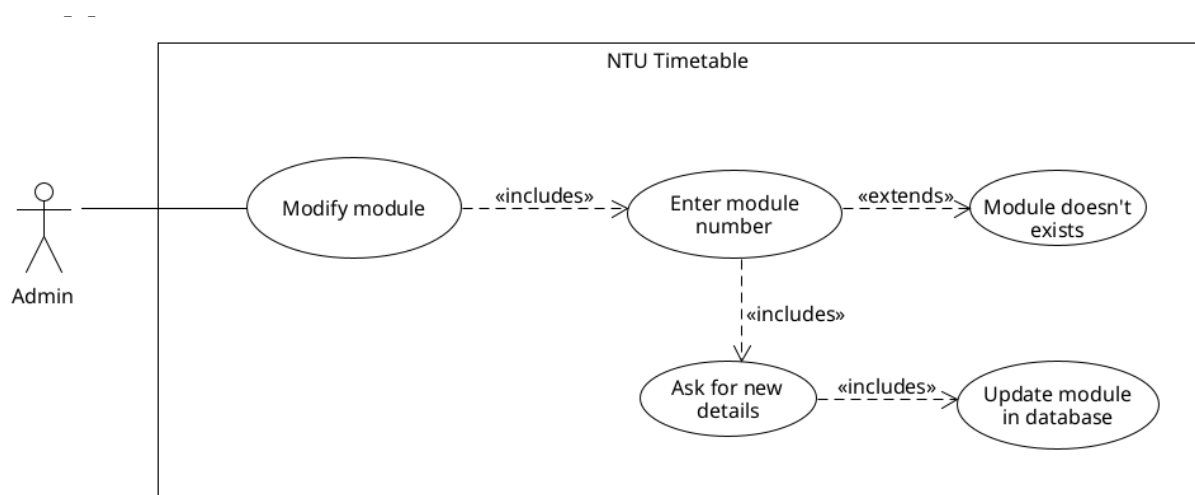
Admin wants to add a module for a course



- Delete module:
Admin wants to delete a module from the system

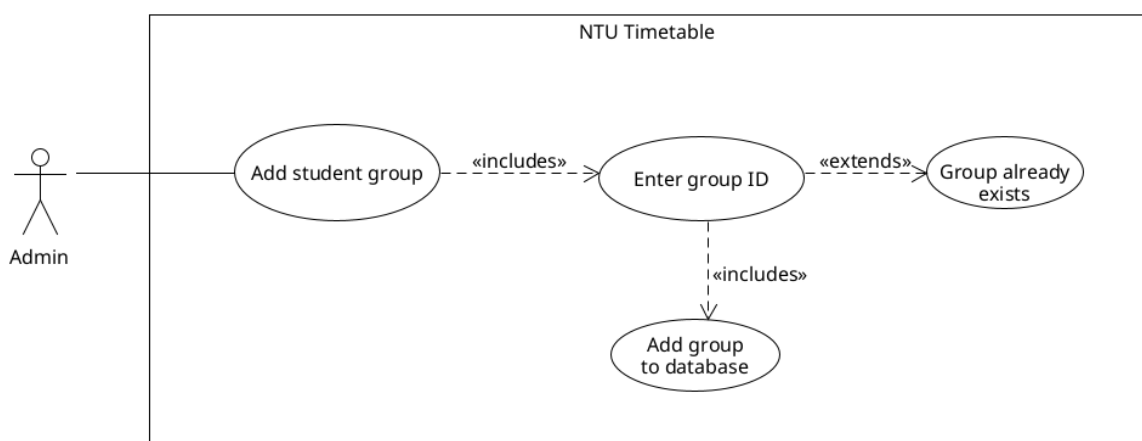


- Modify a module:

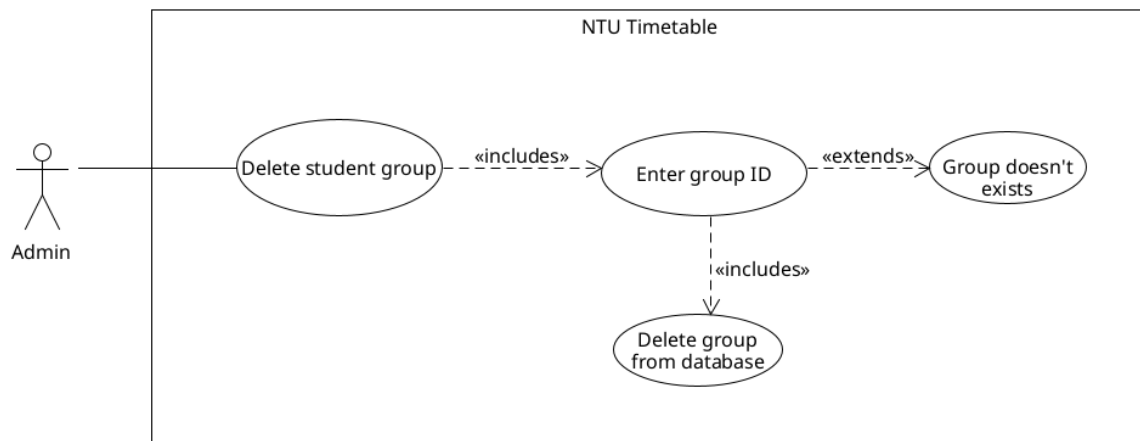


detail about the module

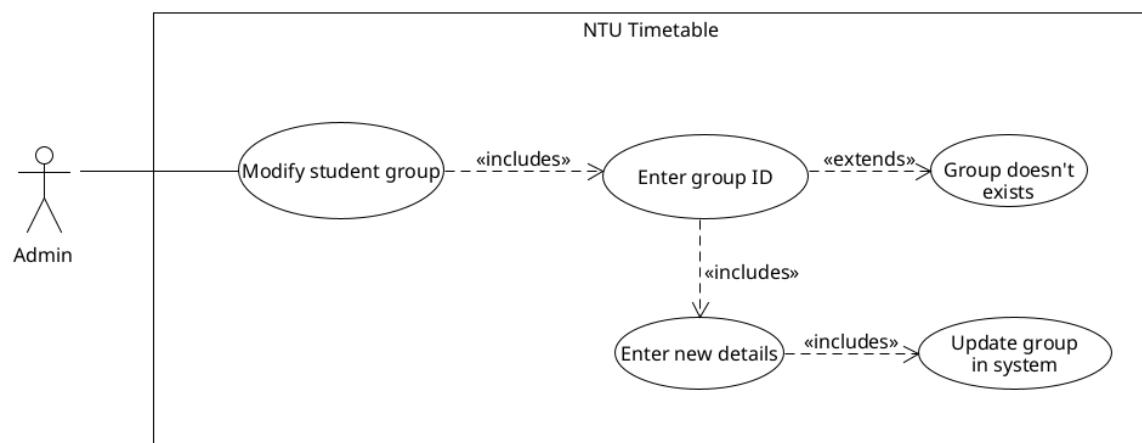
- Add student groups:
Add a new student group into the system



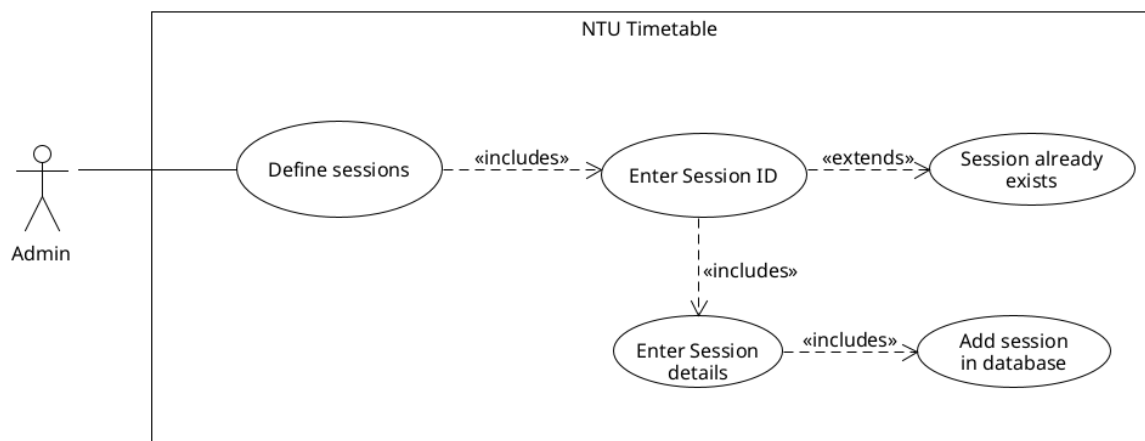
- Delete student groups:
Delete a student group from the data



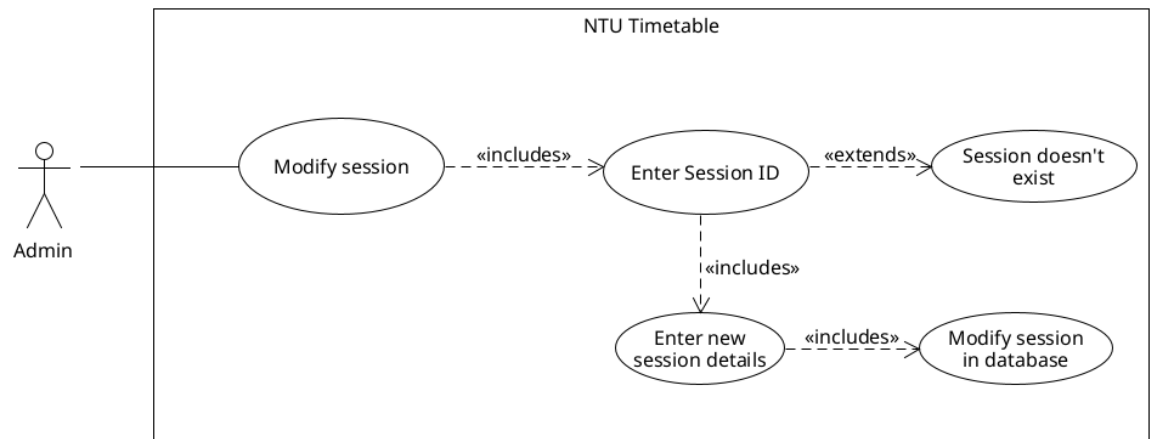
- Modify student group:
Modify group details (course, year, etc.)



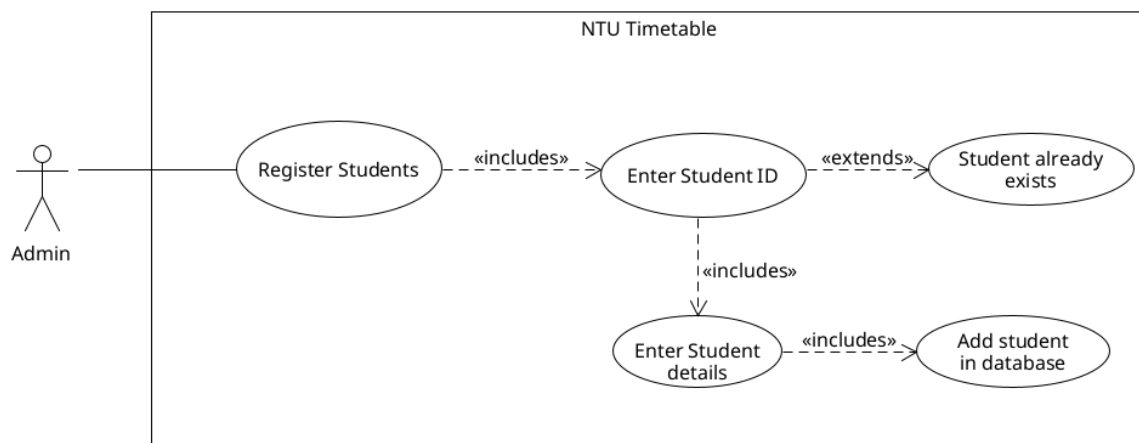
- Define sessions:
Define sessions for an event



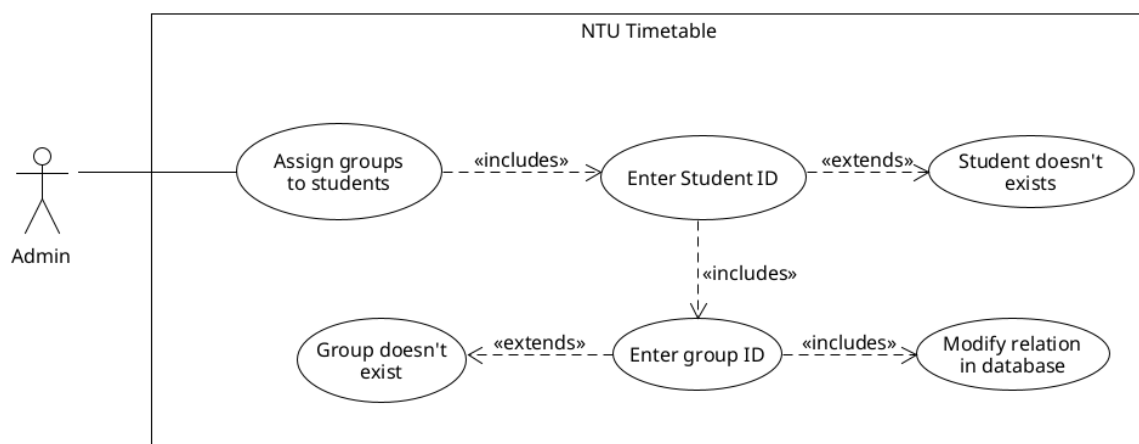
- **Modify session:**
Modify an existing session's



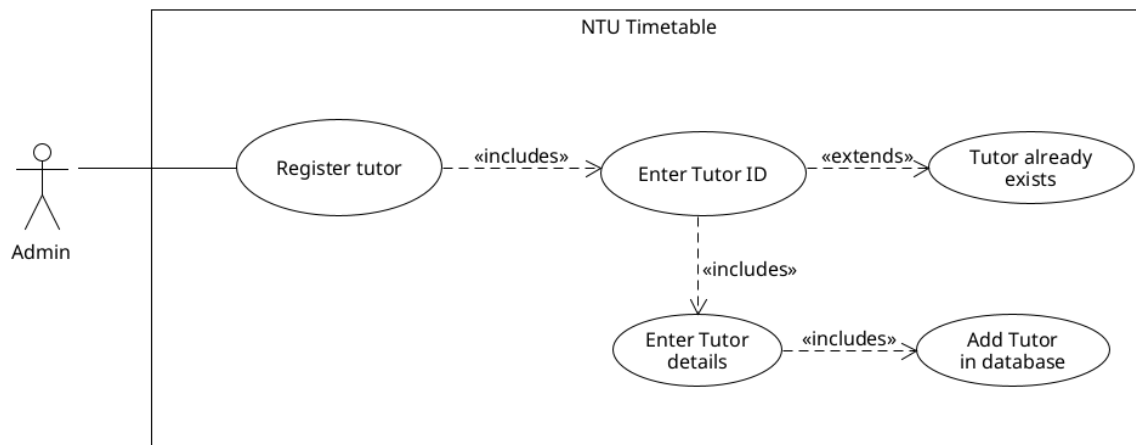
- **Register students:**
Register new students into the system



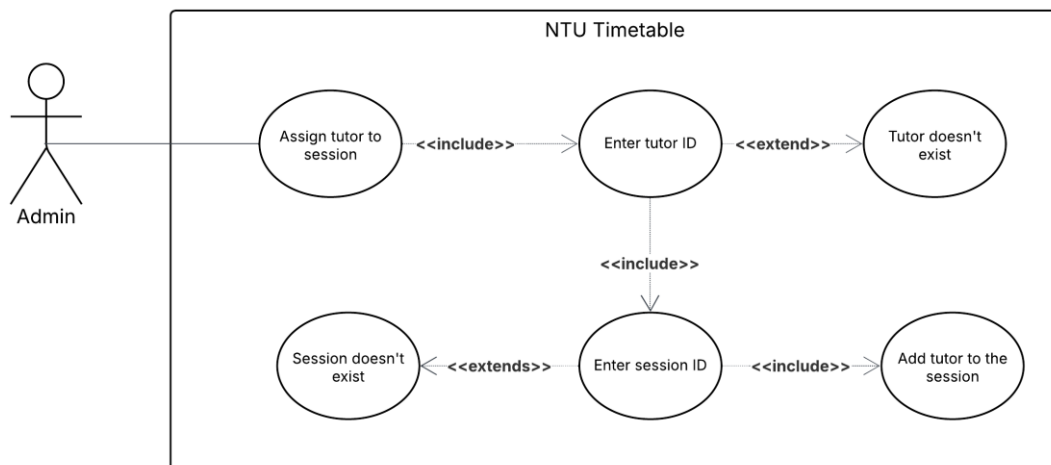
- **Assign student to a student group:**



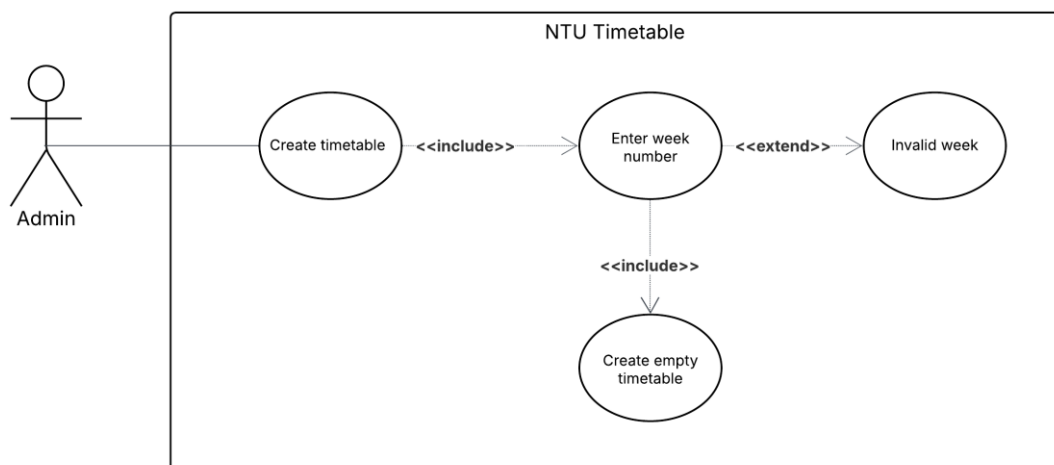
- Register lecturers:
Register new tutors/lecturers



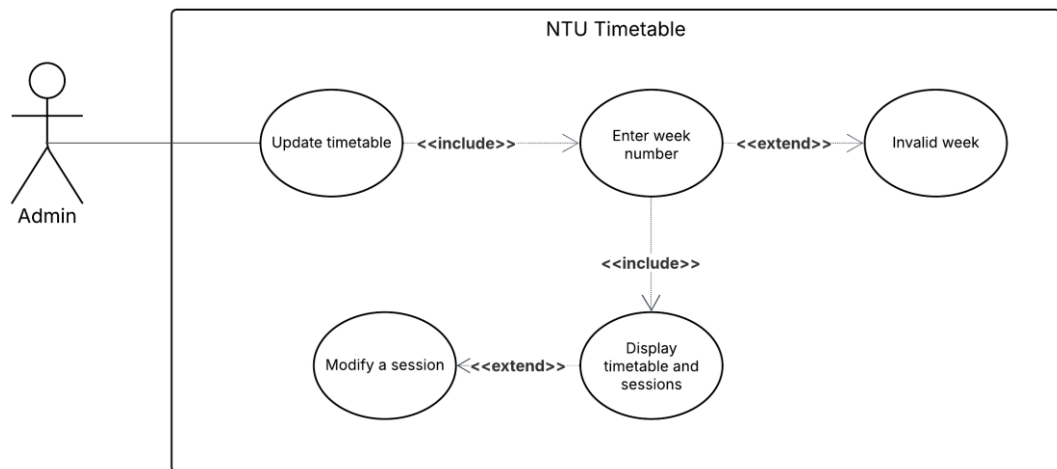
- Assign lecturers and tutors to sessions:
Assign a lecturer to a session



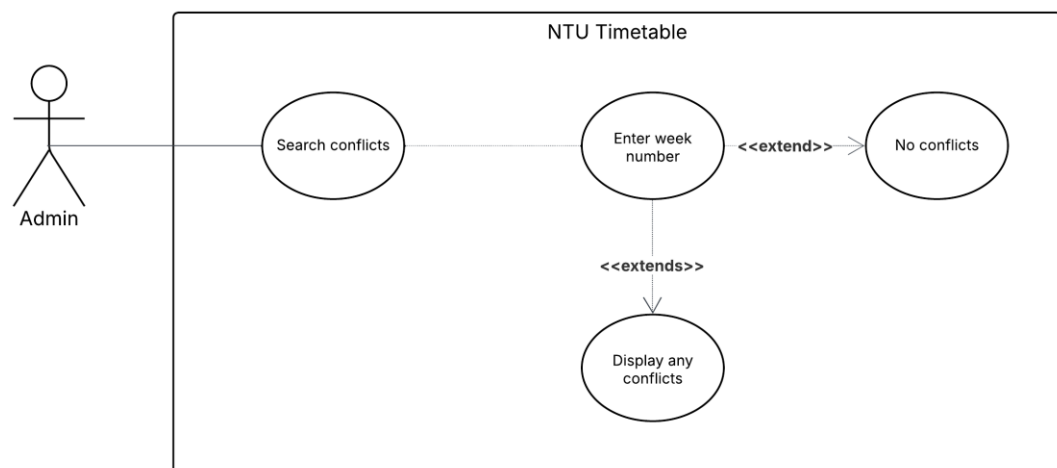
- Create timetable for each week
Create a timetable for a specific week



- Update timetable for a week:
Update timetable for a specific week throughout the term

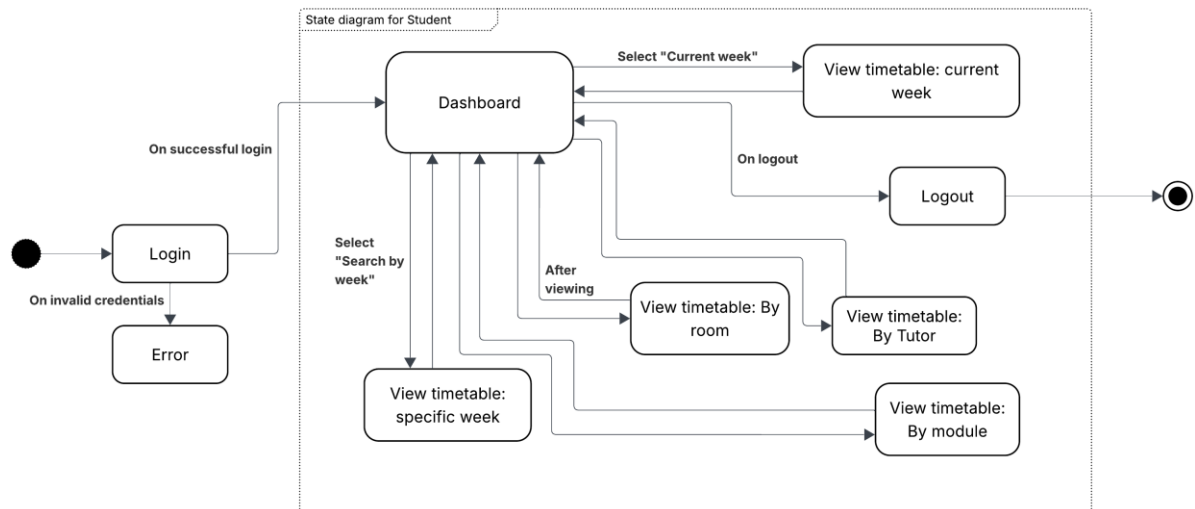


- Search for timetable conflicts:
Look for any conflict between two sessions

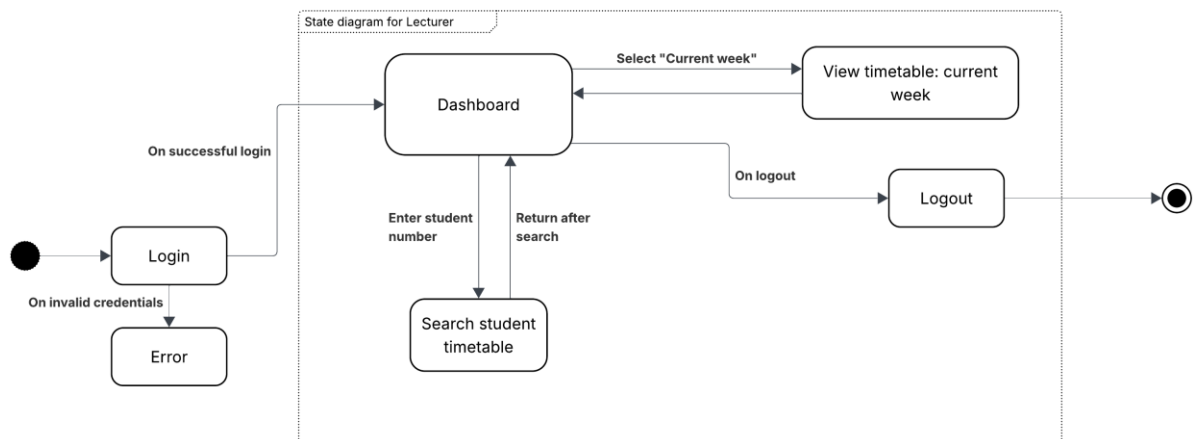


State-Transition Diagram

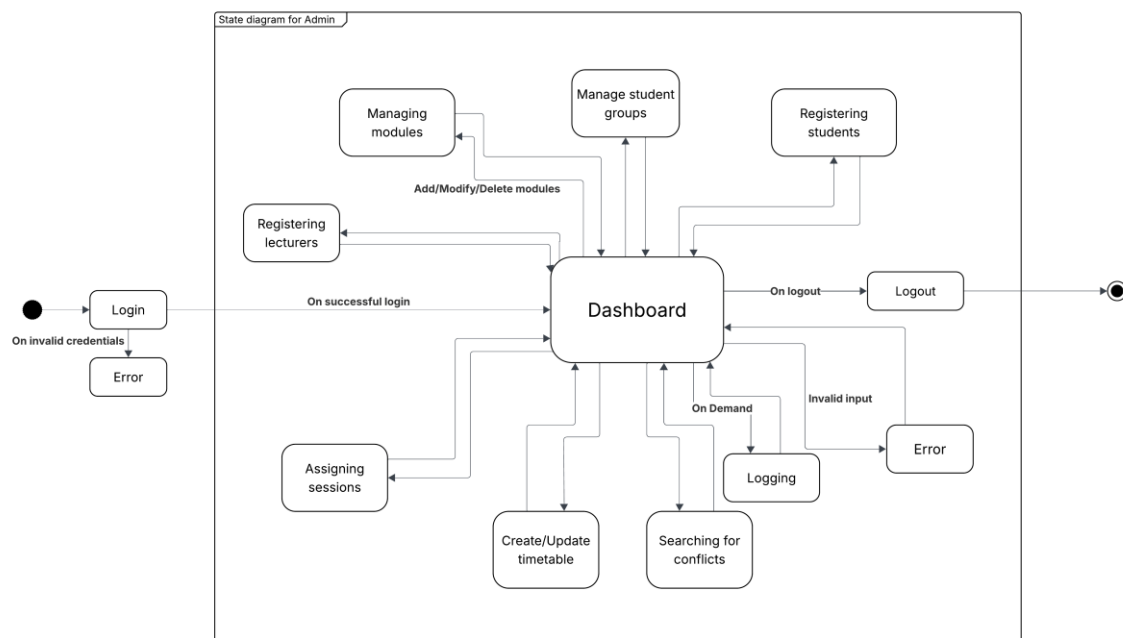
- Student state:



- Lecturer state:

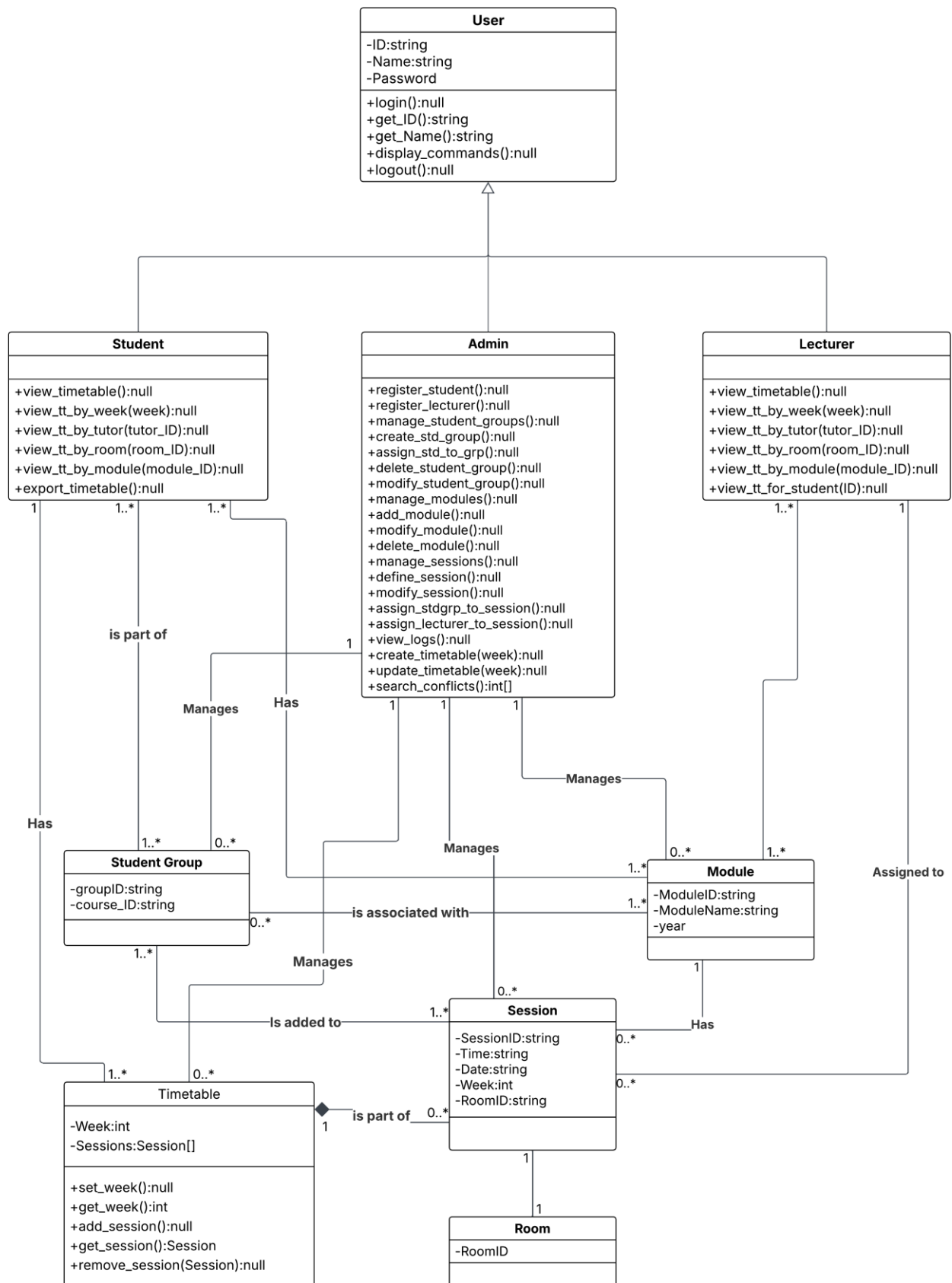


- Admin state:

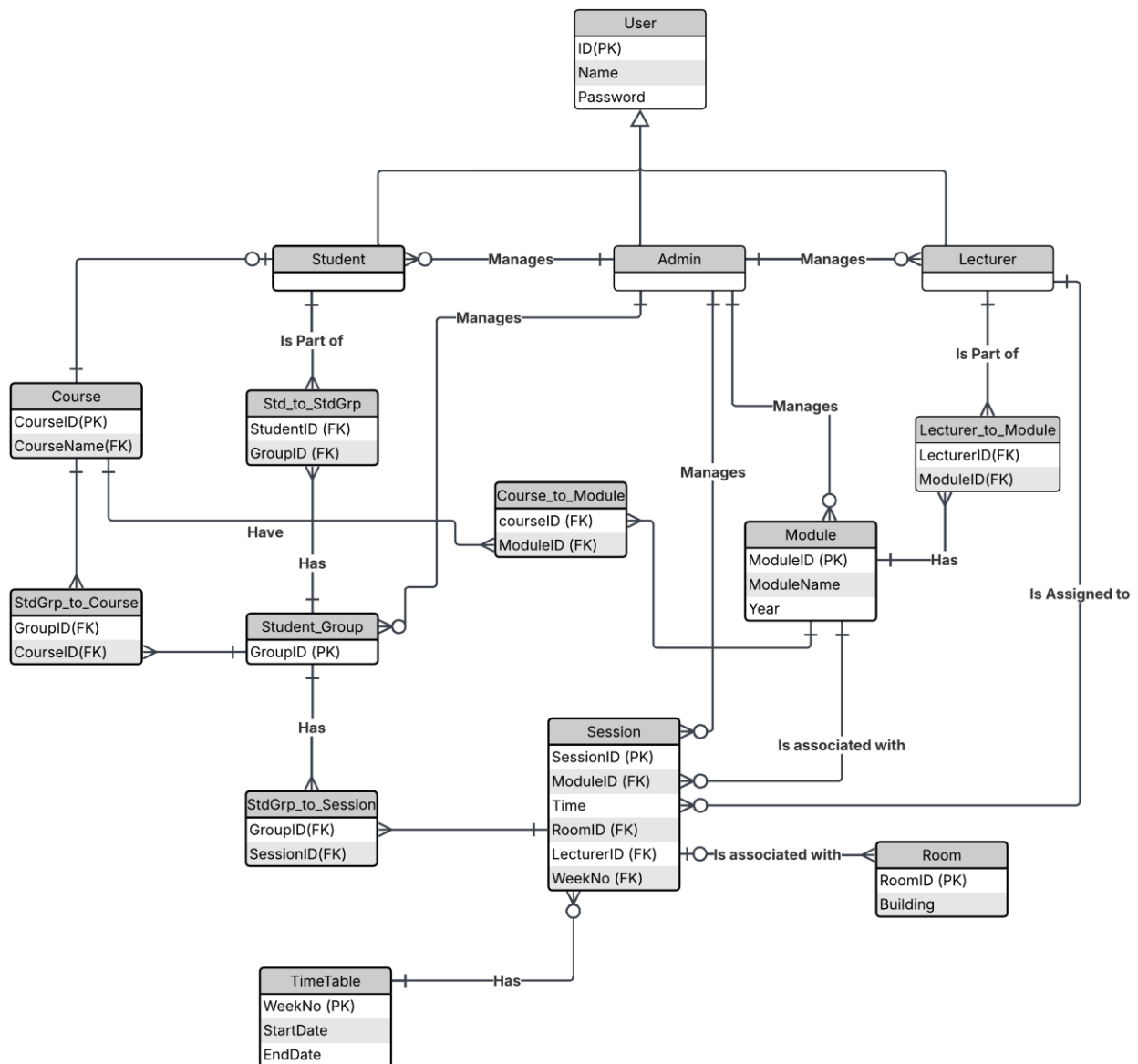


Structural Analysis

Class diagram



Entity Relationship Diagram (ERD)



Critical Analysis

Software Architecture Analysis

The timetabling system is built using a modular, object-oriented architecture. It follows a loosely layered design comprising key components such as:

- **Database Layer** (SQLite): Handles storage and retrieval of data related to students, lecturers, and timetables.
- **Logic Layer**: Manages user authentication, timetable creation, and data validation through classes such as SQL, Timetable, Student, and Lecturer.
- **Interface Layer**: Although the system is console-based, this layer interacts with the user through prompts and displays.

This architecture ensures **separation of concerns**, which enhances **maintainability** and supports future scalability. For example, transitioning to a GUI or a web-based front end would only require changes in the interface layer.

Design Analysis

Several object-oriented and software design principles were followed during development:

- **Abstraction:** Core entities like User, Student, and Lecturer are abstracted as C++ classes, each encapsulating relevant attributes and behavior.
- **Encapsulation:** Data members are kept private, accessed and modified only through public methods. This helps in protecting the integrity of the data.
- **Inheritance:** The Student and Lecturer classes inherit from a base User class, promoting **code reuse** and reducing duplication.
- **Polymorphism:** Functions like displayDetails() are overridden to behave differently based on the actual user type, providing **runtime flexibility**.
- **Single Responsibility Principle:** Each class has a clearly defined role. For instance, the SQL class strictly manages database interaction, whereas the Timetable class focuses solely on scheduling logic.
- **DRY (Don't Repeat Yourself):** Shared operations like database query preparation or input validation are modularized into reusable functions.

By applying these principles, the system maintains a **clean, organized codebase** that is easier to debug, extend, and document.

Implementation Analysis

This section provides a detailed explanation of the core data structures and object-oriented classes used in the development of the simplified NTU Timetabling System. The implementation follows a modular and scalable architecture with clear separation of concerns between user roles, database interaction, and schedule management.

1. User Class (Abstract Base Class)

The User class serves as the abstract base class for all types of users in the system, such as Student and Lecturer.

Attributes:

- **username:** A unique identifier for each user.
- **name:** The full name of the user.
- **password:** Used for authentication.

Purpose:

1. Provides a base structure and common interface for child classes.
2. Enforces inheritance and promotes code reuse.

2. Student and Lecturer Classes (Derived Classes)

Both Student and Lecturer classes inherit from the User class, allowing polymorphic behavior when interacting with different user roles.

Student**Additional Attributes:**

- studentID
- enrolledModules: A list of module IDs the student is enrolled in.

Lecturer**Additional Attributes:**

- lecturerID
- assignedModules: A list of modules assigned for teaching.

3. Module Class

The Module class represents a course or unit offered at NTU.

Attributes:

- moduleCode
- moduleName
- lecturer: Assigned lecturer
- room and timeSlot: For scheduling

Usage:

- Modules are used to build individual timetables for students and lecturers.

4. Schedule / Timetable Class

This class manages the creation, retrieval, and update of schedules.

Responsibilities:

- Maps modules to time slots and rooms.
- Ensures no scheduling conflicts occur.

- Generates personalized views for students and lecturers.

5. SQL Class

Handles all interactions with the SQLite database.

Responsibilities:

- Authenticate users
- Retrieve module and timetable data
- Execute prepared SQL statements

6. Utility Structures

- `std::vector` is used to store collections like modules and schedules.
- `std::map` or `std::unordered_map` is considered for fast lookup of modules and timetable entries.

Evaluation of Non-Functional Requirements

The non-functional requirements define the overall quality and performance attributes expected of the timetabling system. Below is an evaluation of how well the implemented system meets these non-functional requirements:

Performance

- **Expectation:** The system should retrieve and display timetables with minimal latency.
- **Evaluation:** Queries to the SQLite database are optimized using parameterized statements. Timetable retrieval and user login are executed within acceptable timeframes (<1 second under normal load).

Usability

- **Expectation:** Users should be able to navigate and access information intuitively.
- **Evaluation:** The system offers a simple and user-friendly interface through the command-line or minimal GUI (if implemented). Input prompts and error messages are clear and informative.

Security

- **Expectation:** Sensitive information such as passwords must be protected.

- **Evaluation:** Passwords are handled using prepared SQL statements to prevent SQL injection. However, password encryption or hashing has not yet been implemented and should be considered in future iterations.

Scalability

- **Expectation:** The system should accommodate an increasing number of users and classes.
- **Evaluation:** The modular design allows new roles (e.g., Admin) or features (e.g., calendar export) to be added easily. However, scalability is limited by the choice of SQLite, which may not be ideal for large-scale deployment.

Maintainability

- **Expectation:** Codebase should be easy to modify and extend.
- **Evaluation:** The system follows object-oriented principles such as encapsulation and inheritance, which improve maintainability. Logical separation of user roles and data access makes future updates straightforward.

Reliability

- **Expectation:** The system should function consistently without crashing.
- **Evaluation:** Exception handling is implemented during database interactions. The application handles invalid inputs gracefully, reducing the likelihood of runtime errors.

Implementation

Backlog

Id	User Story	Priority	Sprint	Status
PB-01	As a student, I want to log into the system so that I can access my timetable.	High	1	Completed
PB-02	As an admin, I want to log into the system to manage scheduling data.	High	1	Completed
PB-03	As an admin, I want to add and update module information.	Medium	2	Completed
PB-04	As an admin, I want to assign students to modules to generate a timetable.	High	2	Completed

PB-05	As a student, I want to view my timetable with module names and times.	High	3	Completed
PB-06	As an admin, I want to view all module timetables to manage conflicts.	Medium	3	In Progress
PB-07	As a student, I want a simple console-based UI to navigate the system.	Low	4	Completed
PB-08	As a developer, I want to store timetable data in a database to persist data.	High	1	Completed
PB-09	As an admin, I want to search or sort modules/students for quick access.	Medium	4	Not Started
PB-10	As a user, I want proper error messages for failed login attempts	Low	5	Completed

Sprint Planning

Sprint	Duration	User Stories / Tasks	Reason for Inclusion / Priority
Sprint 1	Week 1	- Set up project folder and version control - Design ERD - Create database schema	Foundation tasks for database and architecture planning
Sprint 2	Week 2	- Implement login & registration - Set up base User class and inheritance	Core system access functionality
Sprint 3	Week 3	- Admin features: add modules, assign rooms - Store data in database	Admin roles are critical for timetable generation
Sprint 4	Week 4	- Student view: timetable display - Searching and filtering timetables	Ensures usability for students (main users)
Sprint 5	Week 5	- Testing - Bug fixes - Documentation - Final adjustments and review	Quality assurance and preparation for submission

Implementation Overview

Technologies Used

The development of the Timetabling System leveraged the following tools and technologies:

- **Programming Language:** C++
- **Database:** SQLite for lightweight, file-based relational data storage
- **IDE:** Visual Studio Code / Code::Blocks (specify your actual IDE)
- **Compiler:** g++ (GNU Compiler Collection) for building and running the application
- **Libraries Used:**
 - sqlite3.h for integrating SQLite database functionality
 - Standard Template Library (STL) for vectors, maps, strings, etc

Key Features and Functionalities Developed

1. **User Authentication System**
 - Supports login for both students and lecturers
 - Validates against credentials stored in the database
2. **Role-Based Access**
 - Students can view their timetable
 - Lecturers can view schedules and manage module data
3. **Schedule Management**
 - Retrieves and displays class schedules dynamically from the database
 - Organized by days and modules
4. **Data Validation & Error Handling**
 - Handles invalid inputs, database errors, and missing records gracefully
5. **Object-Oriented Structure**
 - Encapsulation, Inheritance, and Polymorphism applied throughout

Documentation

User

Login/Logout

Purpose:

Enable users to securely access and exit the system.

Instructions:

- **Login:**
 1. Type 0 and press Enter to initiate the login process.

2. Enter your username when prompted and press Enter.
3. Enter your password when prompted and press Enter.
4. If credentials are correct, a success message will appear.

- **Logout:**

1. Type 1 and press Enter to log out.
2. You will receive a confirmation message indicating you have logged out.

Student

View current week's timetable

Purpose:

Allow students to view the timetable for the ongoing week.

Instructions:

- Type 1 on commands page and press Enter.
- The timetable for the current week will be displayed in the console.

View timetable by week

Purpose:

Search and view the timetable for a specific week.

Instructions:

- Type 2 and press Enter.
- Enter the desired week number when prompted and press Enter.
- The timetable for that week will be displayed.

View timetable by room

Purpose:

View sessions in a specific room for the ongoing week.

Instructions:

- Type 3 and press Enter.
- Enter the room number when prompted and press Enter.
- Sessions scheduled in that room will be displayed.

View timetable by lecturer

Purpose:

View sessions with a specific lecturer in the ongoing week.

Instructions:

- Type 4 and press Enter.
- Enter the lecturer's name when prompted and press Enter.
- Sessions with that lecturer will be displayed.

View timetable by module**Purpose:**

View sessions for a specific module in the ongoing week.

Instructions:

- Type 5 and press Enter.
- Enter the module code when prompted and press Enter.
- Sessions for that module will be displayed.

Display weeks**Purpose:**

Print out week numbers and their corresponding dates.

Instructions:

- Type 6 and press Enter.
- A list of weeks with their start and end dates will be displayed.

Lecturer**View Current Week's Timetable****Purpose:**

Allow lecturers to view the timetable for the ongoing week.

Instructions:

- Type 1 and press Enter.
- The timetable for the current week will be displayed in the console.

View Timetable by Week Number**Purpose:**

Search and view the timetable for a specific week.

Instructions:

- Type 2 and press Enter.
- Enter the desired week number when prompted and press Enter.

- The timetable for that week will be displayed.

View Timetable by Room Number

Purpose:

View sessions in a specific room for the ongoing week.

Instructions:

- Type 3 and press Enter.
- Enter the room number when prompted and press Enter.
- Sessions scheduled in that room will be displayed.

View Timetable for a Student

Purpose:

View sessions for a specific student in the ongoing week.

Instructions:

- Type 4 and press Enter.
- Enter the student's ID or name when prompted and press Enter.
- Sessions for that student will be displayed.

Display Weeks and Their Periods

Purpose:

Print out week numbers and their corresponding dates.

Instructions:

- Type 5 and press Enter.
- A list of weeks with their start and end dates will be displayed.

View Timetable by Module

Purpose:

View sessions for a specific module in the ongoing week.

Instructions:

- Type 6 and press Enter.
- Enter the module code when prompted and press Enter.
- Sessions for that module will be displayed.

Admin

Add a new module

Purpose:

Add a new module to the system.

Instructions:

- Type 1 and press Enter.
- Enter the module code and name when prompted and press Enter.
- A confirmation message will indicate the module has been added.

Delete an existing module

Purpose:

Remove a module from the system.

Instructions:

- Type 2 and press Enter.
- Enter the module code when prompted and press Enter.
- A confirmation message will indicate the module has been deleted.

Modify module

Purpose:

Edit the details of an existing module.

Instructions:

- Type 3 and press Enter.
- Enter the module code and new details when prompted and press Enter.
- A confirmation message will indicate the module has been updated.

Add a New Student Group

Purpose:

Create a new student group.

Instructions:

- Type 4 and press Enter.
- Enter the group name and details when prompted and press Enter.
- A confirmation message will indicate the group has been added

Delete a Student Group

Purpose:

Remove a student group from the system.

Instructions:

- Type 5 and press Enter.
- Enter the group name when prompted and press Enter.
- A confirmation message will indicate the group has been deleted.

Modify Student Group Details

Purpose:

Edit the details of an existing student group.

Instructions:

- Type 6 and press Enter.
- Enter the group name and new details when prompted and press Enter.
- A confirmation message will indicate the group has been updated.

Define New Sessions

Purpose:

Set up new sessions such as lectures and labs.

Instructions:

- Type 7 and press Enter.
- Enter the session details when prompted and press Enter.
- A confirmation message will indicate the session has been defined.

Modify Session Details

Purpose:

Edit the details of an existing session.

Instructions:

- Type 8 and press Enter.
- Enter the session ID and new details when prompted and press Enter.
- A confirmation message will indicate the session has been updated.

Register New Students

Purpose:

Add new students to the system.

Instructions:

- Type 9 and press Enter.
- Enter the student ID and name when prompted and press Enter.
- A confirmation message will indicate the student has been registered.

Register a New Course

Purpose:

Add a new course to the system.

Instructions:

- Type 10 and press Enter.
- Enter the course code and name when prompted and press Enter.
- A confirmation message will indicate the course has been registered.

Delete an Existing Course

Purpose:

Remove a course from the system.

Instructions:

- Type 11 and press Enter.
- Enter the course code when prompted and press Enter.
- A confirmation message will indicate the course has been deleted.

Assign Students to Groups

Purpose:

Assign students to specific groups.

Instructions:

- Type 12 and press Enter.
- Enter the student ID and group name when prompted and press Enter.
- A confirmation message will indicate the student has been assigned to the group.

Register Lecturers

Purpose:

Add new lecturers to the system.

Instructions:

- Type 13 and press Enter.
- Enter the lecturer ID and name when prompted and press Enter.
- A confirmation message will indicate the lecturer has been registered.

Assign Lecturers to Sessions

Purpose:

Assign lecturers to specific sessions.

Instructions:

- Type 14 and press Enter.
- Enter the lecturer ID and session ID when prompted and press Enter.
- A confirmation message will indicate the lecturer has been assigned to the session.

Generate Timetable for a Week

Purpose:

Create a timetable for a specific week.

Instructions:

- Type 15 and press Enter.
- Enter the week number and details when prompted and press Enter.
- A confirmation message will indicate the timetable has been created.

Update Existing Timetable

Purpose:

Edit the timetable for a specific week.

Instructions:

- Type 16 and press Enter.
- Enter the week number and new details when prompted and press Enter.
- A confirmation message will indicate the timetable has been updated.

Check Timetable for Conflicts

Purpose:

Identify and display any timetable conflicts.

Instructions:

- Type 17 and press Enter.
- Any conflicts will be displayed in the console.

Conclusion

Summary of the Project Outcomes

The project successfully delivered a functional console-based timetabling system tailored for Nottingham Trent University (NTU). The system achieved its core objectives: enabling administrators to manage modules, student groups, and sessions; allowing students and lecturers to view and search timetables; and implementing conflict detection for overlapping sessions. Key outcomes include:

- A **role-based access system** with distinct functionalities for students, lecturers, and administrators.
- **Dynamic timetable generation** and retrieval using SQLite for data persistence.
- **Conflict detection** to prevent scheduling overlaps.
- A modular, object-oriented architecture adhering to principles like encapsulation, inheritance, and polymorphism, ensuring maintainability and scalability.
- Cross-platform compatibility (Windows, macOS, Linux) and adherence to prioritized functional requirements such as login/logout, timetable search, and data validation.

The system serves as a foundational prototype, demonstrating the feasibility of a scalable backend for future enhancements.

Summary of Challenges Encountered During Development

Several challenges arose during the project's development:

1. **Database Integration:** Initial difficulties in integrating SQLite with C++ required extensive debugging to ensure seamless CRUD operations.
2. **Input Validation:** Handling edge cases for user inputs (e.g., invalid week numbers or module codes) demanded rigorous testing to prevent runtime errors.

3. **Conflict Detection Logic:** Designing algorithms to detect scheduling conflicts while maintaining performance efficiency was complex.
4. **Role-Based Access Control:** Ensuring strict separation of permissions between user roles required careful state management.
5. **Time Constraints:** Balancing feature implementation across sprints led to deferred enhancements, such as password encryption and GUI integration.

Reflection on the Development Process

The adoption of **Scrum methodology** proved advantageous, enabling iterative progress, regular reviews, and adaptive planning. Key takeaways include:

- **Modular Design:** Object-oriented principles and layered architecture simplified debugging and feature expansion.
- **Database Optimization:** Parameterized SQL queries improved performance, though scalability remains limited by SQLite for large datasets.
- **User-Centric Focus:** Prioritizing core functionalities (e.g., timetable viewing) ensured usability, but additional user testing could refine edge cases.
- **Documentation:** Comprehensive documentation streamlined collaboration but highlighted the need for earlier planning of database schemas and use cases.

Overall, the process underscored the importance of balancing technical rigor with pragmatic time management.

Potential Improvements for Future Development

To enhance the system, the following improvements are proposed:

1. **Security Enhancements:** Implement password hashing/encryption to address current vulnerabilities.
2. **Scalability:** Migrate from SQLite to a more robust database system (e.g., PostgreSQL) for larger deployments.
3. **User Interface:** Develop a graphical user interface (GUI) or web-based frontend to improve accessibility.
4. **Advanced Features:**
 - Real-time notifications for timetable updates.
 - Integration with NTU's live databases for data synchronization.
 - Automated conflict resolution algorithms.

5. **Testing:** Conduct stress testing for high user loads and expand unit test coverage.
6. **Accessibility:** Add support for calendar exports (e.g., iCal) and multilingual interfaces.

By addressing these areas, the system could evolve into a comprehensive academic management tool, aligning with NTU's long-term digital strategy.

References

1. SQLite Development Team (2023) *SQLite (Version 3.43.2): C/C++ Interface* [Computer software]. Available at: <https://www.sqlite.org/cintro.html> (Accessed: 25 October 2023).